OmniCare Pathway Engine

1. Executive Summary & Project Overview

The **OmniCare Pathway Engine** is an advanced, AI-driven Clinical Decision Support System (CDSS). At its core, it is an Agentic Multi-Hop GraphRAG (Retrieval-Augmented Generation) application designed to assist healthcare professionals in diagnosing complex or rare diseases, managing comprehensive care pathways, and identifying active clinical trials.

By orchestrating a team of specialized Large Language Model (LLM) agents via LangGraph and grounding their reasoning in a strictly constrained Neo4j medical knowledge graph, OmniCare bridges the gap between unstructured patient data and actionable medical intelligence. It seamlessly integrates foundational diagnostic data (SympScan) with forward-looking experimental protocols (Global Clinical Trials 2024–2026) to provide a holistic, demographic-aware patient care report.

The goal of OmniCare is to build an ultra-smart, AI-powered medical assistant that helps doctors diagnose complex or rare diseases much faster, and immediately connects patients to the most up-to-date treatments and clinical trials.

Why is this needed? (The Problem): When a patient has a rare or complicated illness, doctors have to spend hours reading through messy clinical notes, guessing at overlapping symptoms, checking medical textbooks, and manually searching the internet for experimental treatments. It is a slow process, and for critical patients, time is everything. Furthermore, medical knowledge changes so fast that it's hard for any human to memorize every new clinical trial or research paper.

What does the project actually do? (The Solution): OmniCare takes all that heavy lifting off the doctor's shoulders by doing it automatically in seconds. When a doctor pastes a patient's notes into the system, OmniCare aims to:

1. **Read and Understand:** It reads the messy notes and pulls out the exact symptoms.
2. **Connect the Dots:** It acts like a giant brain (the knowledge graph) to instantly match those symptoms to potential diseases, standard medications, and even diet/exercise plans.
3. **Double-Check the Science:** It browses the internet (PubMed) to make sure it has the absolute latest research on that disease.
4. **Find Hope:** It searches a database of thousands of active clinical trials from around the world to find cutting-edge experimental treatments that fit the patient.
5. **Write the Report:** It bundles all this information into a neat, easy-to-read report tailored specifically to the patient’s age and gender.

The Ultimate Goal: To ensure that no matter how rare a disease is, doctors have instant access to a complete, fact-checked roadmap for patient care—bridging the gap between a confusing list of symptoms and a life-saving clinical trial.

2. The Problem Statement

In modern healthcare, medical professionals face a triad of systemic data challenges when dealing with complex or rare conditions:

- **Unstructured Data Bottlenecks:** Patient symptom histories are often buried within dense, unstructured Electronic Health Record (EHR) clinical notes. Manually extracting actionable data points is time-consuming and prone to human error.
- **Diagnostic Delays & Siloed Knowledge:** Rare diseases often present with overlapping symptoms, leading to misdiagnoses. Furthermore, the knowledge required to map symptoms to diseases, and subsequently to standard medications, dietary plans, and lifestyle precautions, exists in fragmented silos rather than a unified system.
- **Disconnect from Experimental Treatments:** Even when a rare disease is accurately diagnosed, connecting the patient to cutting-edge treatments is difficult. Finding suitable, actively recruiting

clinical trials requires navigating disconnected registries, often resulting in missed opportunities for life-saving interventions.

3. Core Objectives

OmniCare is engineered to solve these bottlenecks through the following primary objectives:

- **Automated Clinical Intake:** To accurately parse and extract standardized medical symptoms from raw, unstructured EHR text using natural language processing.
- **Explainable Diagnostic Routing:** To utilize graph-based reasoning (GraphRAG) that maps symptoms to potential conditions, ensuring every diagnostic prediction is traceable through explicit, multi-hop database relationships.
- **Holistic Care Mapping:** To provide not just a diagnosis, but a complete care pathway that includes standard medications, behavioral precautions, diet, and workout recommendations.
- **Real-Time Research Integration:** To enrich static database knowledge with live validation by scraping the latest abstracts from PubMed based on the predicted condition.
- **Targeted Trial Matching:** To automatically cross-reference predicted diseases with a comprehensive dataset of recent (2024–2026) global clinical trials.
- **Demographic-Aware Synthesis:** To generate a final, readable clinical report that specifically tailors its warnings and recommendations based on the patient's inputted age category and gender.

Results & Outputs

When a clinician successfully runs a patient query through the OmniCare Pathway Engine, the application does not just provide a single prediction; it generates a multi-tiered array of structured data and actionable insights.

The final results presented to the user on the Streamlit dashboard consist of the following key outputs:

1. Standardized Symptom Extraction

- **What is obtained:** A clean, machine-readable, comma-separated list of distinct medical symptoms.
- **Value:** Transforms messy, unstructured, or conversational Electronic Health Record (EHR) notes into structured data points (e.g., converting "Patient complains of head throbbing and feeling very hot" into `["headache", "high fever"]`).

2. Graph-Grounded Diagnostic Matches

- **What is obtained:** A ranked list of the top three most probable diseases based on symptom overlap within the Neo4j Knowledge Graph.
- **Included Data Points:**
    - The specific **Disease Name**.
    - A **Confidence Score** (based on the number of overlapping symptoms).
    - A list of **Standard Medications** conventionally used to manage those specific conditions.

3. Live Research Context

- **What is obtained:** A real-time text summary of recent medical literature.
- **Included Data Points:** The titles of the three most recently published scientific abstracts from PubMed that directly correlate to the patient's primary predicted disease.

4. Targeted Clinical Trial Routing

- **What is obtained:** A curated list of up to five active, globally registered clinical experimental protocols (from the 2024–2026 dataset) that match the patient's condition.
- **Included Data Points:**
    - **NCT ID:** The official registry tracking number (e.g., NCT01234567).
    - **Trial Title:** The official name of the study.
    - **Phase:** The clinical trial phase (e.g., Phase 2, Phase 3).

    - **Status:** Current recruitment status (e.g., Recruiting, Active).

5. The Synthesized Clinical Decision Report (Final Output)

- **What is obtained:** The ultimate culmination of the system's workflow—a comprehensive, formal, and highly readable text report generated by the AI Synthesis Agent.
- **Report Components:**
    - **Differential Diagnoses:** A breakdown of the likely conditions.
    - **Confident Routing Paths:** Clear recommendations on standard care, medications, lifestyle precautions, diet, and workouts (pulled from the SympScan dataset).
    - **Target Trials:** Summaries of the matched experimental interventions.
    - **Demographic-Tailored Warnings:** Crucially, this final report adjusts its language, medication suitability warnings, and trial recommendations specifically based on the **Gender** and **Age Category** (Pediatric, Adult, Geriatric) inputted by the clinician in the UI.

UI Visual Feedback

In addition to the data above, the Streamlit frontend provides immediate visual validation of these results through:

- **Status Badges:** Green/Yellow alert boxes confirming the successful extraction of symptoms and graph resolution.
- **Key Metrics:** A prominent numerical metric displaying the total number of "Active Target Clinical Trials Identified."

4. The Solution: How OmniCare Overcomes the Challenge

OmniCare overcomes the fragmentation of medical data by employing an **Agentic Multi-Hop GraphRAG architecture**. Instead of relying on a single AI model to "guess" a diagnosis, it utilizes a pipeline of specialized agents that execute a deterministic, step-by-step workflow:

1. **Noise Reduction via the Intake Agent:** The system tackles unstructured EHRs by using a dedicated LLM agent to filter out conversational noise, outputting only a clean, machine-readable list of distinct symptoms.
2. **Grounding AI in Reality via Neo4j:** To prevent AI hallucinations, the **GraphRAG Agent** does not rely on the LLM's internal memory. Instead, it queries a highly structured Neo4j graph database (built from the 200+ symptom / 100+ disease SympScan dataset). It traverses the graph from *(Symptom) -> (Disease) -> (Medication/Lifestyle)*, ensuring recommendations are backed by verified data relationships.
3. **Bridging Static and Live Data:** To overcome the limitation of static datasets, the **Scraper Agent** fetches real-time context from PubMed, ensuring the physician is aware of newly published literature regarding the condition.
4. **Automated Experimental Mapping:** The **Trial Matcher Agent** seamlessly connects the diagnosed condition to the Global Clinical Trial Intelligence database. By matching the *(Disease)* node to *(Clinical Trial)* nodes, it instantly provides highly relevant, post-2024 experimental treatment options without requiring the physician to manually search external registries.
5. **Contextual Synthesis:** Finally, the **Synthesis Agent** overcomes the issue of generic AI outputs. By injecting the user-selected demographic data (Age and Gender) into the final prompt alongside the graph and web data, it produces a tailored, highly specific decision support document ready for clinical review.

5. Methodology: How the System is Built

The OmniCare Pathway Engine moves away from traditional, rigid software by using an **Agentic AI Workflow** combined with a **Graph Database**. Instead of writing thousands of lines of "if/then" rules, we built a system that acts like a highly coordinated team of medical researchers.

Here are the three core methodologies powering the engine:

- **Multi-Agent AI (The Specialists):** Instead of using one giant AI model to do everything, the system uses a framework called LangGraph to create a team of specialized "Agents." Each agent has one specific job (e.g., one agent only extracts symptoms, another only reads research papers). They pass their work down the assembly line to the next agent.
- **Graph Database (The Brain's Memory):** Traditional databases store information in flat tables (like Excel). OmniCare uses Neo4j, a *Graph Database*, which stores information like a web. It links a "Symptom" directly to a "Disease," which is then linked to "Medications" and "Clinical Trials." This allows the AI to "connect the dots" instantly, just like human memory.
- **GraphRAG (The Fact-Checker):** RAG stands for *Retrieval-Augmented Generation*. It means the AI is not allowed to guess or make up medical facts. Before the AI answers, it must *retrieve* hard facts from the Neo4j graph database and use those facts to *generate* its response. This eliminates AI hallucinations and ensures clinical safety.

6. Step-by-Step Working: How It Works for the User

When a doctor or healthcare professional uses the OmniCare platform, a seamless, five-step background process is triggered. Think of it as a baton pass in a relay race, where the patient's data is the baton.

Step 1: Patient Intake (The Listener)

- **What Happens:** The doctor pastes a patient's messy, unstructured clinical notes (EHR) into the Streamlit dashboard and selects the patient's age and gender.
- **The AI Action:** The **Intake Agent** reads the notes. It ignores the conversational fluff and pulls out a clean, exact list of symptoms (e.g., "high fever," "joint pain").

Step 2: Diagnostic Matching (The Detective)

- **What Happens:** The extracted symptoms are sent to the Neo4j Knowledge Graph.
- **The AI Action:** The **GraphRAG Agent** searches the database to find which diseases have the highest overlap with those specific symptoms. It identifies the top three most likely conditions and immediately pulls up the standard medications, diets, and physical precautions associated with them.

Step 3: Live Research Scraping (The Academic)

- **What Happens:** Medical knowledge changes daily. The system needs to know if there are any brand-new discoveries about the primary predicted disease.
- **The AI Action:** The **Scraper Agent** takes the top predicted disease and silently browses the internet (PubMed). It reads the titles of the most recently published medical papers and brings that fresh context back to the system.

Step 4: Clinical Trial Routing (The Innovator)

- **What Happens:** If the patient has a rare or stubborn condition, standard medications might not work. The system looks for cutting-edge alternatives.
- **The AI Action:** The **Trial Matcher Agent** dives back into the database, this time looking at the global 2024–2026 Clinical Trials dataset. It finds active, recruiting experimental trials that are specifically targeting the patient's predicted disease.

Step 5: Final Synthesis (The Chief Medical Officer)

- **What Happens:** All the gathered data—the symptoms, the predicted diseases, the standard meds, the live research, and the experimental trials—is handed to the final AI agent.
- **The AI Action:** The **Synthesis Agent** writes a highly professional, easy-to-read Clinical Decision Support Report. It uses the patient's age and gender (selected in Step 1) to tailor the warnings—for example, flagging a medication that might be unsafe for a pediatric patient. This final report is then displayed on the user's screen.

7. Data Architecture & Dataset Utilization

The OmniCare Pathway Engine constructs its intelligence by fusing two distinct data sources into a unified Neo4j Graph Database. The ingestion pipeline (ingest.py) processes seven separate CSV files, transforming flat tabular data into a highly interconnected, multi-hop relational graph.

Below is the detailed breakdown of each dataset file and its specific function within the project.

A. Core Diagnostic Matrix (SympScan)

These files form the foundational reasoning engine for the **GraphRAG Agent**, enabling it to connect patient symptoms to potential medical conditions.

1. Diseases_and_Symptoms_dataset.csv

- **Description:** The primary diagnostic matrix mapping 200+ distinct symptoms to 100+ diagnosable diseases using a binary classification system (1 = present, 0 = absent).
- **Key Columns:** diseases (Target Label), anxiety and nervousness, shortness of breath, chest tightness, etc.
- **How it is used in the project:** * Processed by the ingest_sympscan function.
    - The script iterates through the symptom columns; wherever a 1 is found for a disease, it creates a (Symptom) node and a (disease) node in Neo4j.
    - It establishes the critical (Symptom)-[:INDICATES]->(disease) relationship, which is the exact pathway queried by the GraphRAG Agent to predict conditions.

2. medications.csv

- **Description:** A mapping of standard pharmaceutical treatments recommended for each specific disease.
- **Key Columns:** Disease, Medication (comma-separated list).
- **How it is used in the project:**
    - Processed alongside the symptoms in ingest_sympscan.
    - The script splits the comma-separated string of medicines, creates distinct (Medication) nodes, and links them via the (disease)-[:MANAGED_BY]->(Medication) relationship.

B. Lifestyle & Care Metadata (SympScan)

These files provide the contextual care pathways required by the **Synthesis Agent** to generate holistic, post-diagnosis patient recommendations. They are all processed by the ingest_lifestyle_and_metadata function.

3. descriptions.csv

- **Description:** Contains textual overviews and clinical definitions for every disease in the database.
- **Key Columns:** Disease, Description.
- **How it is used in the project:** * Instead of creating new nodes, this file is used to enrich the existing graph. The script attaches the description as a text property directly onto the corresponding (disease) node (SET d.description = record.desc).

4. precautions.csv

- **Description:** Lists up to four behavioral or environmental precautions a patient should take based on their diagnosis.
- **Key Columns:** Disease, Precaution_1, Precaution_2, Precaution_3, Precaution_4.
- **How it is used in the project:** * The script extracts non-null precaution columns, creates (Precaution) nodes, and maps them using the (disease)-[:HAS_PRECAUTION]->(Precaution) relationship.

5. workout.csv

- **Description:** Suggests physical exercises and activity modifications tailored to specific medical conditions.
- **Key Columns:** Disease, Workouts.
- **How it is used in the project:** * Creates (Workout) nodes and links them via (disease)-[:RECOMMENDS_WORKOUT]->(Workout).

6. diets.csv

- **Description:** Provides dietary guidelines and nutritional plans corresponding to the diagnosed diseases.
- **Key Columns:** Disease, Diet.
- **How it is used in the project:** * Creates (Diet) nodes and links them via (disease)-[:RECOMMENDS_DIET]->(Diet).

C. Experimental Interventions (Global Clinical Trials)

This dataset elevates the system from standard care to cutting-edge medical routing, utilized heavily by the **Trial Matcher Agent**.

7. clinical_trials_2025_2026.csv

- **Description:** A deduplicated, ML-ready dataset of 5,000+ globally registered clinical trials with start dates spanning 2024 to 2028, queried from the ClinicalTrials.gov API.
- **Key Columns:** nct_id, title, status, phase, condition, intervention, brief_summary, country.
- **How it is used in the project:** * Processed by the ingest_clinical_trials function.
    - It creates a central (ClinicalTrial) node identified by its unique nct_id, storing metadata like phase, status, and summary as node properties.
    - It splits the condition column (separated by semicolons) to create the (ClinicalTrial)-[:INVESTIGATES]->(disease) edge, allowing the system to match predicted patient diseases directly to active trials.
    - It parses the intervention column to map the specific experimental treatments being tested via (ClinicalTrial)-[:TESTS]->(Intervention).

Tech Stack

💻 Frontend UI Tier

1. Streamlit

- **Purpose in the Project:** Powers the web-based graphical user interface (app.py), capturing patient demographics (gender, age) and clinical notes, tracking agent execution, and displaying the final clinical decision report.
- **Why It Was Chosen:** Streamlit allows for rapid prototyping of data-driven and AI web applications entirely in Python. It eliminates the need for complex JavaScript/React codebases while natively supporting real-time streaming state feedback, loading indicators, and metric displays.

🚀 API Gateway & Communication Tier

2. FastAPI

- **Purpose in the Project:** Acts as the high-performance core backend framework (server.py) that initializes the server and routes incoming requests.
- **Why It Was Chosen:** FastAPI is one of the fastest Python frameworks available, built on top of Starlette and Pydantic. It provides out-of-the-box asynchronous (asyncio) support, automatic interactive API documentation (/docs via Swagger UI), and strict data validation, which is critical for medical application payloads.

3. LangServe

- **Purpose in the Project:** Connects the FastAPI server to the LangGraph engine (server.py) and allows the frontend client to call the graph remotely (app.py via RemoteRunnable).
- **Why It Was Chosen:** LangServe automatically wraps LangChain and LangGraph objects into REST API endpoints. It saves developers from writing custom wrapper logic for multi-agent applications, providing standardized endpoints for streaming updates, state configurations, and historical invocations.

4. Uvicorn

- **Purpose in the Project:** Serves as the Asynchronous Server Gateway Interface (ASGI) web server that executes the FastAPI application.
- **Why It Was Chosen:** Uvicorn is highly optimized for asynchronous Python workloads, ensuring low overhead and high concurrency handles when routing requests between the interface and the AI engine.

🧠 Cognitive Orchestration Engine (AI Tier)

5. LangGraph

- **Purpose in the Project:** Constructs the structured multi-agent workflow state (StateGraph) that chains the Intake, GraphRAG, Scraper, Trial Matcher, and Synthesis steps sequentially.
- **Why It Was Chosen:** Traditional LLM chains are strictly linear. LangGraph introduces state management, loops, and cycles, enabling developers to build resilient, stateful, multi-agent systems where agents can seamlessly read, update, and pass a shared context (the AgentState object).

6. LangChain Core & Google GenAI

- **Purpose in the Project:** Integrates Google's Gemini models (gemini-3-flash-preview) into the workflow engine using the unified LangChain interface.
- **Why It Was Chosen:** Gemini models provide highly competitive contextual windows, exceptionally fast processing times (perfect for multi-agent loops), and robust extraction capabilities. LangChain Core provides a uniform abstraction layer, making it easy to adjust model hyper-parameters like temperature or swap models entirely in the future.

🗄️ Knowledge Base & Data Processing Tier

7. Neo4j (Graph Database & Driver)

- **Purpose in the Project:** Stores the medical knowledge network containing symptoms, diseases, standard medications, lifestyle properties, and clinical trial records.
- **Why It Was Chosen:** Traditional SQL databases require expensive multi-table joins to trace how a symptom maps to a disease and then to a trial. Neo4j stores relationships as first-class citizens, allowing the GraphRAG agent to execute high-performance, multi-hop Cypher queries instantly.

8. Pandas

- **Purpose in the Project:** Handles the parsing, cleaning, and batching of raw source data from the various CSV files during graph initialization (ingest.py).
- **Why It Was Chosen:** Pandas is the industry standard for tabular data manipulation in Python, enabling quick string cleaning, null value handling, and transformation of large matrix rows into lists for Neo4j consumption via UNWIND optimization clauses.

🌐 External Verification & Web Intelligence

9. BeautifulSoup4 & Requests

- **Purpose in the Project:** Executes live web requests to PubMed and extracts recent abstract HTML components to validate static clinical findings.

- **Why It Was Chosen:** requests provides straightforward HTTP communication, while BeautifulSoup offers an agile HTML parser. Together, they allow the **Scraper Agent** to rapidly bypass API limitations of open medical registries and grab raw text summaries on the fly.

🐳 Infrastructure & Configuration

10. Docker

- **Purpose in the Project:** Packages the entire application execution stack into a single, isolated software container (Dockerfile).
- **Why It Was Chosen:** Containerization ensures that the application behaves identically across local machines, staging environments, and production clouds. It pre-installs required system libraries, sets up health checks, and simplifies orchestration platforms.

11. Python-Dotenv

- **Purpose in the Project:** Dynamically injects secure system variables (API tokens, database credentials) from a local configuration file into runtime environments.
- **Why It Was Chosen:** Decoupling configuration from standard source code is a fundamental software design pillar, keeping sensitive keys out of shared version-controlled code repositories.

8. Technical Challenges & Resolutions

Building a multi-agent, graph-grounded diagnostic engine requires navigating several complex data and architectural hurdles. Below are the primary challenges encountered during development and the strategies implemented to overcome them.

A. LLM Quota Constraints & Deployment Stability

- **The Challenge:** Multi-agent workflows are inherently token-intensive. Having multiple agents (Intake, Scraper, Synthesis) sequentially invoking Large Language Models can quickly lead to API quota exhaustion, rate limiting, and subsequent deployment crashes during active reporting.
- **The Resolution:** To guarantee that the clinical reporting engine remained active and responsive under load, the model architecture was explicitly transitioned to utilize the gemini-3-flash-preview model. This optimized configuration resolved the deployment and quota errors, ensuring high-speed inference without sacrificing the reasoning capabilities required for the final synthesis.

B. Live Web Scraping Volatility

- **The Challenge:** The **Scraper Agent** relies on real-time data from external registries like PubMed. However, live web scraping is notoriously brittle; sudden changes to external HTML DOM structures, network latency, or temporary server blocks can cause the agent to fail, breaking the entire LangGraph pipeline.
- **The Resolution:** Defensive programming was implemented within the research_scraper_agent. A strict 5-second timeout was applied to the requests.get call, wrapped in a comprehensive try/except block. If PubMed is unreachable or the HTML structure changes, the agent gracefully degrades by returning a fallback string ("Live validation scraping window currently unavailable") rather than crashing the workflow.

C. Graph Ontology & Data Normalization

- **The Challenge:** Merging distinct, semi-structured datasets (SympScan's binary matrix and the Global Clinical Trials textual API dumps) into a unified Neo4j database presented significant data cleaning hurdles. Inconsistent casing, trailing whitespace, and un-normalized lists could result in duplicate nodes or disconnected graph edges.
- **The Resolution:** The ingest.py script was designed with rigorous string normalization pipelines. Techniques such as stripping whitespace, forcing lowercase matching during Cypher queries (toLower(s.name) IN $symptoms), and implementing strict database constraints (e.g., REQUIRE

d.name IS UNIQUE) were enforced prior to ingestion. This guarantees a single source of truth and prevents graph fragmentation.

D. Managing Context Window & State Handoff

- **The Challenge:** In a sequential LangGraph setup, the AgentState dictionary continually grows as each agent appends its findings. If not managed carefully, passing raw unstructured EHR notes, JSON arrays of trials, and scraped text into the final **Synthesis Agent** could overwhelm the model's context window or lead to prompt confusion.
- **The Resolution:** The system strictly defines the AgentState using Python's TypedDict, ensuring rigid data contracts between agents. The **Intake Agent** specifically reduces the unstructured EHR noise into a concise, comma-separated list *before* it is passed down the chain. This prevents downstream agents from processing unnecessary conversational fluff, keeping the final prompt clean and targeted.

9. System Design & Component Architecture

The OmniCare Pathway Engine is built on a decoupled, microservice-inspired architecture. By separating the data ingestion, the AI orchestration, the API gateway, and the frontend interface into distinct modules, the system remains highly scalable and maintainable.

Below is the detailed breakdown of the four core files that power the application and how they interact.

A. ingest.py (The Data Foundation)

Role: The ETL (Extract, Transform, Load) script responsible for constructing the Neo4j Knowledge Graph from raw CSV datasets. It is run once (or on scheduled updates) before the main application starts.

How it Works:

1. **Connection & Constraints:** It initializes a connection to the Neo4j database using the official Python driver. The init_constraints() method applies strict uniqueness rules (e.g., REQUIRE d.name IS UNIQUE) to prevent duplicate nodes during ingestion.
2. **Batch Processing (Pandas):** It uses pandas to read the large medical and clinical trial CSVs. Instead of writing complex for-loops that insert records one by one, it transforms the dataframe rows into a list of Python dictionaries.
3. **Cypher UNWIND Operations:** It passes these lists to Neo4j using the UNWIND Cypher clause. This is a highly optimized batch-insertion technique that allows the database to process thousands of nodes and relationships (like (Symptom)-[:INDICATES]->(disease)) in a single transaction, significantly reducing ingestion time.

B. agents.py (The Cognitive Brain)

Role: This is the core intelligence module. It defines the LangGraph state machine, the individual LLM agents, and the specific prompts used to query Gemini and Neo4j.

How it Works:

1. **State Definition (AgentState):** It strictly defines the data structure passing between agents using Python's TypedDict. This ensures that fields like raw_notes, matched_diseases, and final_report are consistently updated and available to downstream agents.
2. **Agent Methods:** The WorkflowEngine class encapsulates the agents.
    - *LLM Agents* (Intake, Synthesis) use self.llm.invoke(prompt) to process language.
    - *Graph Agents* (GraphRAG, Trial Matcher) bypass the LLM and directly query Neo4j using self.driver.session().run() based on the variables currently in the state.
    - *Tool Agents* (Scraper) use requests and BeautifulSoup to fetch external data.
3. **Graph Compilation (compile_graph):** It uses LangGraph's StateGraph to map the agents into a sequential pipeline. It defines the entry point (intake), connects the nodes via edges (add_edge), and compiles them into a single, executable AI workflow.

C. server.py (The API Gateway)

Role: The backend server that exposes the compiled LangGraph engine as a network-accessible REST API.

How it Works:

1. **FastAPI Initialization:** It creates a high-performance FastAPI application instance, defining the core metadata and routing.
2. **Engine Instantiation:** It imports the WorkflowEngine from agents.py and compiles the graph (app_graph).
3. **LangServe Wrapping:** Instead of manually writing POST endpoints to handle the complex JSON inputs and outputs of the LangGraph, it uses LangServe's add_routes(). This automatically wraps the app_graph into standardized, production-ready endpoints (e.g., /omnicare/invoke, /omnicare/stream) and mounts them to the FastAPI app.
4. **Execution:** It runs via the uvicorn ASGI server, listening for incoming requests on port 8000.

D. app.py (The User Interface)

Role: The frontend client that interacts with the user, collects inputs, and visually renders the AI's diagnostic workflow and final report.

How it Works:

1. **UI Construction:** It uses streamlit to quickly render an interactive dashboard, including layout columns, select boxes for demographics (Age, Gender), and a text area for the EHR notes.
2. **Remote Connection:** It uses LangServe's RemoteRunnable("http://localhost:8000/omnicare") to create a seamless client-side connection to the FastAPI server. To the Streamlit app, calling the remote API feels exactly like calling a local Python function.
3. **Execution & Tracking:** When the user clicks the submit button, it packs the UI data into the initial AgentState dictionary and sends it to the API via app_graph.invoke(initial_state).
4. **Rendering:** Upon receiving the completed state from the server, it parses the dictionary, updates the UI metrics (like the number of active trials found), and renders the final_report using Streamlit's markdown engine.

End-to-End Data Flow Summary

To summarize the system design, here is the lifecycle of a single user request:

1. **User** pastes notes and selects demographics in app.py.
2. app.py sends this payload over HTTP to server.py.
3. server.py passes the payload into the compiled LangGraph defined in agents.py.
4. The agents inside agents.py sequentially process the data, querying the Neo4j database (populated by ingest.py) and scraping PubMed.
5. The final synthesized text is returned to server.py, which sends it back over HTTP to app.py, where it is displayed to the **User**.

10. End-to-End System Execution Flow

The OmniCare Pathway Engine processes a user query through a highly orchestrated, state-driven pipeline. From the moment a clinician clicks "Execute," the initial data payload—known as the AgentState—is sequentially enriched by AI models, graph databases, and external APIs.

Here is the detailed, step-by-step journey of a single user request.

Phase 1: User Input & Initialization (Frontend)

1. **Data Capture (app.py):** The clinician inputs the patient's demographics (e.g., "Pediatric", "Female") via dropdown menus and pastes unstructured clinical notes into the Streamlit text area.

2. **State Creation:** Streamlit packages this information into the initial AgentState dictionary. At this stage, fields like raw_notes, gender, and age_category are populated, while downstream fields (like extracted_symptoms and matched_diseases) are initialized as empty lists or strings.
3. **API Dispatch:** Streamlit uses LangServe's RemoteRunnable to send an HTTP POST request containing this initial state to the FastAPI backend.

Phase 2: Request Orchestration (Backend)

4. **Endpoint Reception (server.py):** The FastAPI server receives the payload at the /omnicare/invoke endpoint.
5. **Graph Trigger:** The LangServe wrapper automatically passes the payload into the compiled WorkflowEngine (the LangGraph StateGraph), triggering the entry point: the **Intake Agent**.

Phase 3: The Multi-Agent Pipeline (agents.py)

This is where the core cognitive processing occurs. The AgentState is passed like a baton through five distinct nodes. At each node, an agent performs a specific task and *appends* its findings to the state.

- **Node 1: Intake Agent**
  - *Action:* Invokes Gemini to read the raw_notes and strip away conversational noise.
  - *State Update:* Injects a clean list of medical terms into the extracted_symptoms array.
- **Node 2: GraphRAG Agent**
  - *Action:* Takes the extracted_symptoms and executes a multi-hop Cypher query against the Neo4j database to find overlapping conditions and their standard treatments.
  - *State Update:* Appends a list of dictionaries (containing disease names, confidence scores, and medications) into matched_diseases.
- **Node 3: Scraper Agent**
  - *Action:* Extracts the top primary disease from the state and sends a live HTTP request to PubMed, scraping the HTML for the latest published abstracts.
  - *State Update:* Injects the scraped text into the web_enrichment string.
- **Node 4: Trial Matcher Agent**
  - *Action:* Takes the matched_diseases and queries the Neo4j trial database to find active clinical trials (from the 2024–2026 dataset) targeting those specific conditions.
  - *State Update:* Populates the clinical_trials array with trial metadata (NCT IDs, phase, status).
- **Node 5: Synthesis Agent**
  - *Action:* The final Gemini invocation. It takes the *entire* enriched state (Demographics, Symptoms, Diseases, Meds, PubMed data, and Trials) and is prompted to write a formal, tailored clinical report.
  - *State Update:* Generates the final markdown text and saves it to final_report.

Phase 4: Delivery (Frontend)

6. **Return Payload:** The LangGraph execution finishes (END node), and FastAPI sends the fully populated AgentState back to the Streamlit client over HTTP.
7. **UI Rendering (app.py):** Streamlit unpacks the final state. It updates the UI metrics (e.g., displaying the number of trials found, showing the extracted symptoms) and renders the final_report using markdown format for the clinician to review.

The Value of LangSmith Integration

- **Visualizing the Graph:** LangSmith provides a visual UI of the LangGraph execution, allowing developers to see exactly how the `AgentState` payload changes as it moves from the Intake Agent down to the Synthesis Agent.
- **Cost & Token Monitoring:** It automatically logs the token count for every Gemini API call, ensuring the system stays within rate limits.
- **Debugging Live Scrapes:** If the **Scraper Agent** fails due to a PubMed HTML change, LangSmith traces exactly what context was lost and how the fallback error was handled by downstream agents.